



Insider Threat Detection Using Deception Technology

A senior project submitted in partial fulfillment of the requirements for the degree of Bachelor of Computers and Artificial Intelligence.

“Information Security & Digital Forensics” Program,

Project Team

- 1- Ahmed Sherif Ali
- 2- Marwan Sherif Kamel
- 3- Ahmed Emad Nasr
- 4- Abdelrahman Sayed Shweal
- 5- Joseph George Nessim
- 6- Karim Tarek Ali

Under Supervision of

Dr. Ahmed Tawfik
Eng. Maryam Moaz

Benha, **August 2025**

DECLARATION

We hereby certify that this material, which we now submit for assessment on the program of study leading to the award of Bachelor of Computers and Artificial Intelligence in *Information Security & Digital Forensics* is entirely our own work, that we have exercised reasonable care to ensure that the work is original, and does not to the best of our knowledge breach any law of copyright, and has not been taken from the work of others save and to the extent that such work has been cited and acknowledged within the text of our work.

Signed: _____

Signed: _____

Signed: _____

Signed: _____

Signed: _____

Signed: _____

Date:

M O N T H - Y E A R

ABSTRACT

This project presents a deception-based insider threat detection system designed to identify malicious activities performed by trusted users within organizational environments. Traditional security solutions primarily focus on external attackers and often fail to detect insider threats, as insiders already possess legitimate access to systems and sensitive information. This creates a critical security gap, where malicious actions can remain unnoticed until significant damage has occurred.

The proposed system addresses this challenge by deploying deceptive components such as fake files, fake credentials, and honeytokens that appear legitimate but hold no real operational value. Any interaction with these deceptive assets is treated as a high-confidence indicator of malicious insider behavior. The system follows a modular architecture that separates deceptive asset management, interaction monitoring, threat detection, and logging and alerting functions. Deceptive assets are strategically created, deployed, and maintained within the environment, while all access attempts are continuously monitored and analyzed.

Once an interaction with a deceptive element is detected, the system validates the asset, confirms the threat, and generates detailed security events. These events are logged and forwarded to a centralized Security Information and Event Management (SIEM) platform to support correlation, auditing, and incident response. Real-time alerts are also sent to security analysts to enable rapid investigation and mitigation. By treating deceptive asset interaction as a definitive signal rather than relying on complex behavioral analysis, the system reduces false positives and increases detection confidence.

This project demonstrates how deception technologies can significantly enhance insider threat detection by improving visibility, accuracy, and response time. The proposed approach provides a practical and scalable solution for modern organizations seeking to protect sensitive assets from internal misuse while maintaining normal user operations and minimizing performance overhead.

TABLE OF CONTENTS

TABLE OF CONTENTS	III
TABLE OF FIGURES	V
LIST OF TABLES	VI
LIST OF ACRONYMS/ABBREVIATIONS	1
CHAPTER ONE	2
1 INTRODUCTION	2
1.1 OVERVIEW OF THE DECEPTION-BASED INSIDER THREAT DETECTION SYSTEM.....	2
1.2 PROBLEM STATEMENT & SIGNIFICANCE.....	2
1.3 OBJECTIVES.....	3
1.4 SCOPE OF WORK.....	3
1.5 METHODOLOGY OVERVIEW	4
1.6 SIGNIFICANCE OF THE PROJECT.....	5
1.7 METHODOLOGIES	5
1.7.1 Methodological Framework (Agile/Scrum)	5
1.8 SOFTWARE DEVELOPMENT LIFE CYCLE (SDLC) MODEL.....	6
1.8.1 Phase 1: Requirements Elicitation	7
1.8.2 Phase 2: Architectural & Component-Level Design.....	7
1.8.3 Phase 3: Development and Implementation	8
1.8.4 Phase 4: Verification, Validation & Security Testing	8
1.8.5 Phase 5: Sprint Review & Feedback	9
1.9 WORK BREAKDOWN STRUCTURE (WBS)	9
1.9.1 Project Management.....	9
1.9.2 System Architecture & Design	9
1.9.3 Backend Development	10
1.9.4 Frontend Development	10
1.9.5 Quality Assurance & Security Validation	10
1.9.6 Deployment & Operations.....	10
1.10 SCHEDULE ALLOCATION.....	11
1.11 RESOURCE ALLOCATION.....	11
1.12 BUDGET ALLOCATION.....	12
1.13 RISK MANAGEMENT & QUALITY ASSURANCE	12
1.13.1 Risk Management	12
1.13.2 Quality Assurance (QA)	12
1.14 TECHNIQUES & TOOLS.....	13
1.15 STRUCTURE OF THE DOCUMENT.....	13
1.16 SUMMARY.....	14
CHAPTER TWO	15
2 BACKGROUND AND LITERATURE REVIEW	15
2.1 INSIDER THREAT DETECTION USING DECEPTION TECHNIQUES	15
2.2 HONEYTOKENS AND CANARY-BASED DETECTION MODELS.....	15
2.3 INSIDER THREAT DETECTION THROUGH USER BEHAVIOR ANALYSIS.....	16
2.4 HONEYPOTS AND DECOY SYSTEMS FOR INSIDER MONITORINGDESPITE ADVANCES IN CYBERSECURITY, CURRENT SYSTEMS SHOW SEVERAL GA PS:.....	16
2.5 DECEPTION-BASED SECURITY FRAMEWORKS FOR ZERO TRUST ENVIRONMENTS.....	17
2.6 INTEGRATION OF SIEM AND DECEPTION TECHNOLOGIES	17
2.7 LIMITATIONS OF EXISTING APPROACHES.....	17

2.8	SUMMARY AND RESEARCH GAP	18
CHAPTER 3		19
3	SYSTEM DESIGN & IMPLEMENTATION	19
3.1	SYSTEM OVERVIEW	19
3.2	BLOCK DIAGRAM.....	20
3.2.1	<i>Insider Threat Detection System Using Deceptive Techniques -Block Diagram</i>	21
3.3	CONTEXT DIAGRAM.....	22
3.3.1	<i>Insider Threat Detection System Using Deceptive Techniques- Context Diagram</i>	23
3.4	DATA FLOW DIAGRAM – LEVEL 0.....	24
3.4.1	<i>Insider Threat Detection System Using Deceptive Techniques -DFD Level 0 Diagram</i>	25
3.5	DATA FLOW DIAGRAM – LEVEL 1.....	26
3.5.1	<i>DFD Level 1 – Process 2: Deception Asset Interaction</i>	26
3.5.2	<i>Insider Threat Detection System Using Deceptive Techniques -Block Diagram DFD Level 1 – Process 2: Deception Asset Interaction</i>	27
3.5.3	<i>DFD Level 1 – Process 3: Insider Behavior Analysis</i>	28
3.5.4	<i>Insider Threat Detection System Using Deceptive Techniques -DFD Process 3 Diagram</i>	29
3.5.5	<i>DFD Level 1 – Process 4: Honeypot Redirection & Logging</i>	30
3.5.6	<i>Insider Threat Detection System Using Deceptive Techniques -DFD Process 4 Diagram</i>	31
3.6	ACTIVITY DIAGRAM	32
3.6.1	<i>Insider Threat Detection System Using Deceptive Techniques -Activity Diagram</i>	33
3.7	SEQUENCE DIAGRAM.....	34
3.7.1	<i>Insider Threat Detection System Using Deceptive Techniques - Sequence Diagram</i>	35
3.8	USE CASE DIAGRAM.....	36
3.8.1	<i>Insider Threat Detection System Using Deceptive Techniques – Use Case Diagram</i>	37
3.9	GANTT CHART	38
3.9.1	<i>Insider Threat Detection System Using Deceptive Techniques -Activity Diagram</i>	39
3.10	CHAPTER SUMMARY.....	40
CHAPTER FOUR		41
4	EXPERIMENTAL RESULTS AND FUTURE WORK	41
4.1	FUNCTIONAL TESTING.....	41
4.1.1	<i>Insider Threat Detection Workflow</i>	41
4.1.2	<i>Deceptive Asset Deployment Validation</i>	42
4.1.3	<i>Logging and Alert Verification</i>	43
4.2	NON-FUNCTIONAL TESTING.....	44
4.2.1	<i>Performance & Scalability Testing</i>	44
4.2.2	<i>Security Validation</i>	45
4.3	HOLISTIC EVALUATION.....	45
4.3.1	<i>Cost Analysis</i>	46
4.3.2	<i>Ethical Considerations</i>	46
4.3.3	<i>Operational Impact & Safety</i>	46
4.3.4	<i>Sustainability</i>	47
4.4	FUTURE WORK.....	48
4.5	DEPLOYMENT IN REAL ORGANIZATIONAL ENVIRONMENTS.....	48
4.6	ADVANCED INSIDER BEHAVIOR PROFILING.....	48
4.7	INTEGRATION WITH SIEM AND SOC PLATFORMS.....	49
4.8	MACHINE LEARNING–BASED THREAT CLASSIFICATION.....	49
4.9	EXPANSION OF DECEPTIVE ASSETS PORTFOLIO.....	49
4.10	AUTOMATED RESPONSE AND CONTAINMENT MECHANISMS.....	49
4.11	PERFORMANCE OPTIMIZATION AND SCALABILITY TESTING.....	50
4.12	COMPLIANCE AND POLICY CUSTOMIZATION.....	50
4.13	CHAPTER SUMMARY.....	50
5	REFERENCES	51

TABLE OF FIGURES

FIGURE 3-1 BLOCK DIAGRAM.....21

FIGURE 3-2 CONTEXT DIAGRAM.....23

FIGURE 3-3 DFD LEVEL 0 DIAGRAM.....25

FIGURE 3-4 BLOCK DIAGRAM DFD LEVEL 1 – PROCESS 2: DECEPTION ASSET INTERACTION.....27

FIGURE 3-5 BLOCK DIAGRAM DFD LEVEL 1 – PROCESS 2: DECEPTION ASSET INTERACTION.....27

FIGURE 3-6 DFD PROCESS 3 DIAGRAM.....29

FIGURE 3-7 DFD PROCESS 4 DIAGRAM.....31

FIGURE 3-8 ACTIVITY DIAGRAM34

FIGURE 3-9 SEQUENCE DIAGRAM.....35

FIGURE 3-10 USE CASE DIAGRAM.....37

FIGURE 3-11 ACTIVITY DIAGRAM.....39

LIST OF TABLES

TABLE 4-1 TEST CASES AND RESULTS FOR DECEPTION-BASED INSIDER THREAT DETECTION SYSTEM.....42

TABLE 4-2 VERIFICATION OF DECEPTIVE ASSET DEPLOYMENT AND MONITORING.....42

TABLE 4-3 DETECTION ENGINE TEST RESULTS.....43

TABLE 4-4 SYSTEM PERFORMANCE TEST RESULTS.....44

TABLE 4-5 SECURITY TEST RESULTS45

TABLE 4-6 SYSTEM COMPONENT COST ESTIMATES46

LIST OF ACRONYMS/ABBREVIATIONS

Acronyms	Definition
AES	Advanced Encryption Standard
API	Application Programming Interface
CI/CD	Continuous Integration / Continuous Deployment
DFD	Data Flow Diagram
ERD	Entity Relationship Diagram
QA	Quality Assurance
RSA	Rivest–Shamir–Adleman (public-key cryptography)
SDLC	Software Development Life Cycle
SIEM	Security Information and Event Management
SOC	Security Operations Center
SRS	Software Requirements Specification
UBA	User Behavior Analytics
UI	User Interface
UML	Unified Modeling Language
WBS	Work Breakdown Structure
ZTS	Zero Trust Architecture

CHAPTER ONE

1 INTRODUCTION

1.1 OVERVIEW OF THE DECEPTION-BASED INSIDER THREAT DETECTION SYSTEM

With the rapid expansion of digital infrastructures and the growing dependence on information systems, organizations are increasingly exposed to security threats originating from within their own networks. Insider threats are particularly challenging because insiders often hold **legitimate access privileges** and possess in-depth knowledge of internal systems. These advantages allow malicious insiders to bypass traditional security controls and operate undetected.

This project presents a **deception-based insider threat detection system** aimed at identifying and mitigating malicious activities carried out by trusted users. The system utilizes **deceptive assets** such as fake files, fake credentials, and honeytokens that appear authentic but hold no operational value. These assets are **strategically deployed** within the organizational environment to lure malicious insiders without affecting normal user operations.

Any interaction with these deceptive elements is treated as a **high-confidence indicator of malicious intent**, which immediately triggers **detection and response mechanisms** to prevent potential damage and ensure system security.

1.2 PROBLEM STATEMENT & SIGNIFICANCE

Traditional security solutions primarily focus on defending against **external threats**, leaving organizations vulnerable to attacks originating from **trusted insiders**. Malicious insiders may exploit their legitimate access to sensitive information, intellectual property,

or critical system functions, often going undetected by conventional perimeter-based defenses.

The significance of this project lies in its ability to address this gap by providing a **systematic and proactive method** for detecting insider threats. By leveraging deception technologies, organizations can monitor and analyze interactions with fake assets to **identify suspicious behavior early**, thereby reducing potential financial, operational, and reputational losses. The system not only detects threats but also **provides actionable intelligence** regarding the identity, intent, and activities of malicious insiders.

1.3 OBJECTIVES

The main objectives of this project are as follows:

1. **Develop a deception-based framework** to detect insider threats effectively.
 2. **Deploy fake assets** such as files, credentials, and honeytokens within the environment.
 3. **Monitor interactions** with deceptive assets and capture detailed event data.
 4. **Analyze user behavior** to determine whether interactions indicate malicious intent.
 5. **Integrate alerting and logging mechanisms** to notify security analysts and feed SIEM systems.
 6. **Evaluate the effectiveness** of the detection system in identifying insider threats while minimizing disruption to normal operations.
-

1.4 SCOPE OF WORK

The scope of this project includes:

- Designing and implementing **deceptive assets** to attract potential malicious insiders.
 - Developing the **interaction monitoring and detection engine** to analyze access attempts.
 - Implementing **logging and alerting mechanisms** to ensure timely notification of threats.
 - Testing the system in a controlled environment to measure **detection accuracy, responsiveness, and reliability**.
 - Excluding internet-based information or external threat vectors, focusing solely on **internal security risks**.
-

1.5 METHODOLOGY OVERVIEW

The methodology followed in this project includes:

1. **Design Phase** – Define the types of deceptive assets, their placement, and the monitoring framework.
2. **Implementation Phase** – Develop the detection engine and deploy deceptive assets within a simulated environment.
3. **Data Collection** – Capture user interactions with deceptive assets, recording all events for analysis.
4. **Analysis Phase** – Evaluate interactions against detection rules to identify potential insider threats.
5. **Alerting and Reporting** – Generate logs, send alerts to SIEM, and notify security analysts in real time.
6. **Evaluation and Reflection** – Assess system performance, analyze results, and document lessons learned.

1.6 SIGNIFICANCE OF THE PROJECT

This project contributes to the field of **cybersecurity and insider threat detection** by:

- Providing organizations with a **proactive approach** to identify malicious insiders.
 - Enhancing understanding of insider behaviors through **interaction analysis**.
 - Offering a **modular and scalable solution** that can be integrated with existing monitoring systems such as SIEM platforms.
 - Supporting **decision-making processes** for security administrators by supplying actionable insights and detailed reports.
-

1.7 METHODOLOGIES

1.7.1 Methodological Framework (Agile/Scrum)

The development of the Deception-Based Insider Threat Detection System follows an **Agile Scrum methodology**, due to the iterative nature of the project, the need for continuous testing, and evolving requirements in insider threat detection. Agile allows incremental development cycles, **continuous feedback**, and flexible adaptation of system components.

Reasons for Choosing Agile/Scrum

- Rapid response to changes in threat modeling and detection requirements.
- Continuous feedback from supervisors and security analysts.
- Modular development structure: **Frontend, Backend, Deception Engine, Logging/Monitoring**.

- Supports **continuous integration and deployment** of new detection rules and monitoring modules.

Scrum Roles

Role	Responsibilities
Product Owner	Defines requirements, sets priorities, validates increments.
Scrum Master	Ensures process compliance, removes blockers.
Development Team	Implements backend, frontend, monitoring engine, and alerting modules.

Scrum Artifacts

- **Product Backlog:** All features including deceptive asset deployment, detection engine, alerting, and logging.
- **Sprint Backlog:** Tasks assigned for each 2-week sprint.
- **Increment:** Deliverable at the end of each sprint (e.g., deployed honeytokens, functional detection engine).

1.8 SOFTWARE DEVELOPMENT LIFE CYCLE (SDLC) MODEL

The project follows a **hybrid SDLC** combining Agile and **Secure SDLC principles**:

1. **Agile** – Iterative development and continuous feedback.
2. **Secure SDLC** – Ensures proper implementation of monitoring, logging, and data handling.

1.8.1 Phase 1: Requirements Elicitation

Functional and non-functional requirements were collected from:

- Supervisor interviews
- Review of insider threat detection literature
- Analysis of existing security solutions
- Security analyst feedback

Deliverables:

- SRS Document (Functional & Non-Functional)
- Use Case Descriptions
- Requirements Traceability Matrix
- Initial System Architecture

1.8.2 Phase 2: Architectural & Component-Level Design

Main components designed:

- **Deceptive Asset Module:** Generates and deploys fake files, credentials, and honeytokens.
- **Detection Engine:** Monitors asset interaction and flags suspicious behavior.
- **Logging & Alerting Module:** Records events, sends notifications, and integrates with SIEM.
- **Central Repository:** Stores metadata and monitoring data.

Deliverables:

- Architectural Diagrams
- Sequence Diagrams

- DFDs (Level 0 & Level 1)
- ERD for storage of events and assets

1.8.3 Phase 3: Development and Implementation

Development tracks:

1. **Frontend Development:** Dashboards for monitoring interactions, access approval UI, alert notifications.
2. **Backend Development:** Detection engine logic, logging APIs, alert triggers.
3. **Integration:** Connect detection engine with fake assets, logging module, and SIEM interface.

Deliverables:

- Functional dashboards
- Detection engine operational
- Logging & alerting system integrated

1.8.4 Phase 4: Verification, Validation & Security Testing

Testing types:

- **Functional:** Detection of unauthorized access to fake assets.
- **Security:** Verification of logging integrity, SIEM alerts, unauthorized access detection.
- **Performance:** Response time for real-time alerting (<2 seconds).

Deliverables:

- Test Cases and Reports
- Vulnerability Assessment
- Performance Evaluation

1.8.5 Phase 5: Sprint Review & Feedback

- Prototype demonstrated to supervisors and security analysts
 - Feedback integrated into the next sprint
 - Continuous alignment with project objectives and insider threat detection requirements
-

1.9 WORK BREAKDOWN STRUCTURE (WBS)

1.9.1 Project Management

- Requirement gathering
- Milestone planning
- Sprint planning & retrospectives
- Risk management
- Documentation (Proposal, Mid-term, Final report)

1.9.2 System Architecture & Design

- Overall system architecture
- Deceptive asset deployment structure
- Detection engine workflow

- Logging & alerting design
- UML/DFD diagrams

1.9.3 Backend Development

- Detection engine module
- Logging & alerting API
- SIEM integration module
- Error handling & middleware

1.9.4 Frontend Development

- Dashboards for analysts
- Asset interaction visualizations
- Alert notifications interface

1.9.5 Quality Assurance & Security Validation

- Unit and integration testing
- Detection engine validation
- Alerting system verification
- Performance and usability testing

1.9.6 Deployment & Operations

- Docker containerization of backend and detection engine
- SIEM integration setup
- Logging database deployment
- Continuous monitoring setup

1.10 SCHEDULE ALLOCATION

6-Month Academic Timeline

Month Milestone

- 1–2 Requirements gathering, backend setup
 - 3 Detection engine development
 - 4 Frontend dashboards and alerting
 - 5 Integration and testing
 - 6 Final deployment, documentation, and defense
-

1.11 RESOURCE ALLOCATION

Technical Resources

- Python / Django
 - React / Tailwind CSS
 - Docker / CI-CD pipelines
 - SIEM integration tools
 - GitHub / VS Code
-

1.12 BUDGET ALLOCATION

Item	Estimated Cost
Cloud hosting (optional)	\$10–30/month
SIEM tools (optional)	\$20–50/month
Domain & certificates	\$10–15/year
Misc. hardware & peripherals	\$50–100

1.13 RISK MANAGEMENT & QUALITY ASSURANCE

1.13.1 Risk Management

Risk	Impact	Mitigation
Logging system downtime	High	Redundant logging servers
False positives in detection	Medium	Continuous refinement of rules
Unauthorized access	High	AES encryption, role-based access control
Integration errors	Medium	Modular architecture, extensive testing

1.13.2 Quality Assurance (QA)

- Test-driven development
- Automated backend and detection engine tests
- Performance monitoring
- Usability evaluation for dashboards

- Logging and alerting verification
-

1.14 TECHNIQUES & TOOLS

- **Frontend Tools**
 - React, Tailwind CSS, React Router, Fetch
- **Backend Tools**
 - Python, Django, Django REST Framework, AES/RSA, Logging APIs
- **Integration Tools**
 - SIEM connectors, Event monitoring, Alerting modules
- **DevOps Tools**

Docker, GitHub Actions, CI/CD pipelines, Local monitoring setup

1.15 STRUCTURE OF THE DOCUMENT

This document is structured to provide a **clear, consistent, and comprehensive** guide for understanding the project:

1. **Chapter 1 – Introduction:** Provides overview, problem statement, objectives, scope, and methodology.
2. **Chapter 2 – Literature Review:** Reviews existing research, frameworks, and techniques related to insider threat detection.
3. **Chapter 3 – System Design & Implementation:** Details the architecture, design decisions, and technical implementation of the deception-based detection system.

4. **Chapter 4 – Experimental Results and Future Work** : This chapter presents experimental results and performance analysis, summarizes key conclusions, and outlines future improvements and research directions.
 5. **References**: Lists all sources cited in the document following standard academic citation guidelines.
-

1.16 SUMMARY

Chapter One introduces the deception-based insider threat detection system, explaining the problem of insider threats and the limitations of traditional security solutions. It outlines the project objectives, scope, and methodology, highlighting the use of deceptive assets to detect malicious behavior. The chapter also presents the development approach, system design planning, risk management, and overall document structure, providing a foundation for the rest of the report.

CHAPTER TWO

2 BACKGROUND AND LITERATURE REVIEW

2.1 INSIDER THREAT DETECTION USING DECEPTION TECHNIQUES

One of the earliest and most influential research directions in insider threat detection is the use of deception as a proactive defense mechanism. In this context, deception refers to the deliberate deployment of misleading information or resources to detect malicious behavior. In [1], the authors introduced the concept of using deceptive files and decoy documents within enterprise systems to detect insider threats. These decoy files were designed to appear highly sensitive and valuable, such as financial reports or confidential project plans. The study demonstrated that legitimate users rarely interact with such files during normal operations, while malicious insiders are highly likely to access them during reconnaissance or data exfiltration attempts. The researchers concluded that interactions with deceptive files provide high-confidence indicators of malicious intent with very low false-positive rates. This study strongly supports the core idea of the proposed system, which relies on fake files and honeytokens as primary detection triggers. [1]

2.2 HONEYTOKENS AND CANARY-BASED DETECTION MODELS

Honeytokens are digital artifacts that serve no legitimate operational purpose but are monitored to detect unauthorized access or misuse. In [2], the authors explored the use of honeytokens embedded within databases, file systems, and credential stores. These honeytokens included fake user accounts, API keys, and database records. Any attempt to use or access these tokens resulted in immediate alerts.

The study highlighted that honeytokens are particularly effective against insiders because they exploit curiosity and opportunistic behavior. Unlike external attackers, insiders often have contextual knowledge that allows them to identify high-value assets. By planting realistic honeytokens among real assets, defenders can reliably detect suspicious behavior. This approach aligns directly with the proposed project, which deploys fake credentials and deceptive assets across multiple system layers. [2] **Insider Threat Detection Models**

2.3 INSIDER THREAT DETECTION THROUGH USER BEHAVIOR ANALYSIS

Several studies have focused on behavioral analysis as a method for detecting insider threats. In [3], the authors proposed a user behavior analytics (UBA) framework that monitors user actions such as file access patterns, login times, and command execution frequency. Machine learning models were used to establish baseline behavior profiles for each user and detect deviations.

While the results showed promising detection accuracy, the study acknowledged significant limitations, including high computational overhead and susceptibility to false positives caused by legitimate behavioral changes. Compared to behavioral-only approaches, deception-based systems offer more deterministic detection signals. The proposed project addresses this limitation by combining deception triggers with contextual logging, reducing reliance on purely statistical anomaly detection. [3]

2.4 HONEYPOTS AND DECOY SYSTEMS FOR INSIDER MONITORING

DESPITE ADVANCES IN CYBERSECURITY, CURRENT SYSTEMS SHOW SEVERAL GAP S:

Traditional honeypots have long been used to detect external attackers; however, their application in insider threat detection has gained attention in recent years. In [4], the researchers deployed internal honeypots designed to mimic critical servers and services within an organization. These internal honeypots were invisible to legitimate workflows but attractive to insiders attempting lateral movement or privilege escalation.

The study demonstrated that internal honeypots significantly improved detection of malicious insiders during early attack stages. However, the authors noted challenges related to deployment complexity and maintenance. The proposed system addresses these challenges by focusing on lightweight deceptive assets, such as fake files and credentials, rather than full-scale honeypot environments. [4]

2.5 DECEPTION-BASED SECURITY FRAMEWORKS FOR ZERO TRUST ENVIRONMENT SYSTEM ARCHITECTURE

In [5], the authors proposed a deception-driven security framework integrated into a Zero Trust Architecture (ZTA). The framework continuously verified user trust levels and dynamically deployed deceptive resources based on risk scores. High-risk users were presented with more deceptive assets, increasing the likelihood of detection.

2.6 INTEGRATION OF SIEM AND DECEPTION TECHNOLOGIES

Security Information and Event Management (SIEM) platforms play a critical role in correlating and analyzing security events. In [6], the researchers examined the integration of deception technologies with SIEM systems to enhance insider threat detection. Deception events were assigned higher severity scores and correlated with user activity logs.

The results showed that deception-generated alerts were more actionable and required less manual investigation compared to traditional alerts. This finding directly supports the integration of the proposed system with monitoring and logging solutions such as Wazuh or SIEM platforms. The project adopts a similar philosophy by treating interactions with deceptive assets as high-confidence security incidents. [6]

2.7 LIMITATIONS OF EXISTING APPROACHES

Despite the effectiveness of deception-based techniques, existing studies highlight several limitations. These include the need for realistic asset design, the risk of attackers identifying deceptive elements, and the lack of standardized deployment methodologies. In [7], the authors emphasized that poorly designed deception systems may lose effectiveness over time if insiders become aware of their presence.

The proposed project addresses these concerns by focusing on realism, minimal operational impact, and modular deployment. By continuously refining deceptive assets

and integrating them naturally into the environment, the system aims to maintain long-term effectiveness. [7]

2.8 SUMMARY AND RESEARCH GAP

Based on the reviewed studies, it is evident that deception-based techniques offer a powerful and reliable approach to insider threat detection. However, many existing solutions are either too complex, resource-intensive, or tightly coupled to specific enterprise environments.

The research gap identified through this review lies in the need for a lightweight, modular, and easily deployable deception-based insider threat detection system that can be integrated with existing monitoring tools. The proposed project directly addresses this gap by combining fake files, fake credentials, and honeytokens into a unified detection framework. This approach builds upon prior research while introducing practical design and implementation considerations suitable for real-world environments.

CHAPTER 3

3 SYSTEM DESIGN & IMPLEMENTATION

This chapter presents the detailed system design and implementation structure of the proposed deception-based insider threat detection system. It focuses on illustrating how the system components interact, how data flows across the environment, and how insider activities are monitored and analyzed through well-defined architectural and behavioral models. A set of standardized diagrams is used to provide a clear and structured representation of the system, ensuring both technical clarity and alignment with software engineering best practices.

3.1 SYSTEM OVERVIEW

The proposed system is designed to detect insider threats by leveraging deception techniques such as honeytokens, fake assets, and controlled redirection to honeypots. The architecture integrates monitoring, detection, logging, and response mechanisms to identify malicious insider behavior without disrupting legitimate users.

The design emphasizes:

- Clear separation of concerns
- Secure data flow between entities and processes
- Traceability and auditability of insider actions
- Scalability and extensibility for future enhancements

3.2 BLOCK DIAGRAM

The Block Diagram provides a high-level architectural view of the system and illustrates the main components involved in the deception-based insider threat detection solution. It focuses on the structural organization of the system rather than the detailed workflow.

The system is composed of multiple interconnected blocks, including the User Interface, Authentication and Access Control Module, Deception Assets Repository, Monitoring and Logging Module, and the Security Analysis Engine. Legitimate users interact with the system through the user interface, while insiders attempting unauthorized access are silently redirected toward deceptive assets. All interactions are continuously monitored and logged for further analysis.

This diagram highlights the separation of concerns between system components and demonstrates how security monitoring is embedded within the architecture without affecting normal system operations.

Key Components:

- Insider User (Legitimate or Malicious)
- Authentication & Authorization Module
- Deception Engine (Fake Assets & Honeytokens)
- Monitoring & Detection Engine
- Honeypot Environment
- Logging & Forensic Storage

- Security Administrator Dashboard

3.2.1 Insider Threat Detection System Using Deceptive Techniques -Block Diagram

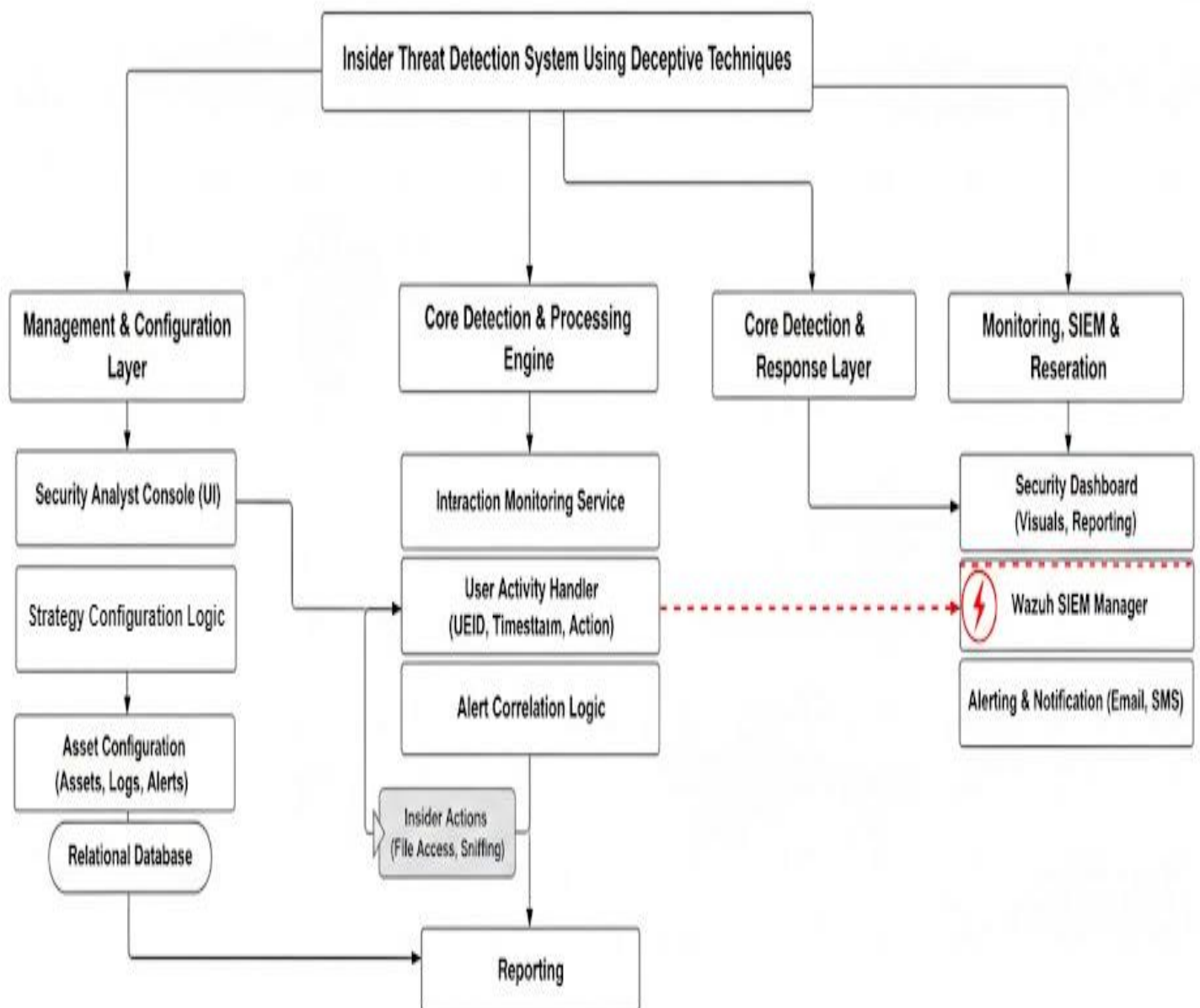


Figure 3-1 Block Diagram

3.3 CONTEXT DIAGRAM

The Context Diagram presents a system-level view that defines the system boundary and its interaction with external entities. It shows the system as a single process and focuses on how external actors communicate with it.

In this diagram, external entities such as Authorized Users, Malicious Insiders, System Administrators, and the Security Operations Team interact with the system through defined data flows. Authorized users access legitimate system resources, while malicious insiders may unknowingly interact with deceptive assets deployed within the environment.

The Context Diagram emphasizes that all interactions, whether legitimate or malicious, pass through the system boundary, enabling centralized monitoring, detection, and response. This diagram serves as the foundation for more detailed data flow diagrams presented in subsequent sections.

.

External Entities Include:

- Insider User
- System Administrator
- Organizational Information Systems
- Incident Response Team

The diagram shows the flow of requests, alerts, logs, and responses between the system and its surrounding environment, providing a clear understanding of the system's scope and responsibilities.

3.3.1 Insider Threat Detection System Using Deceptive Techniques- Context Diagram

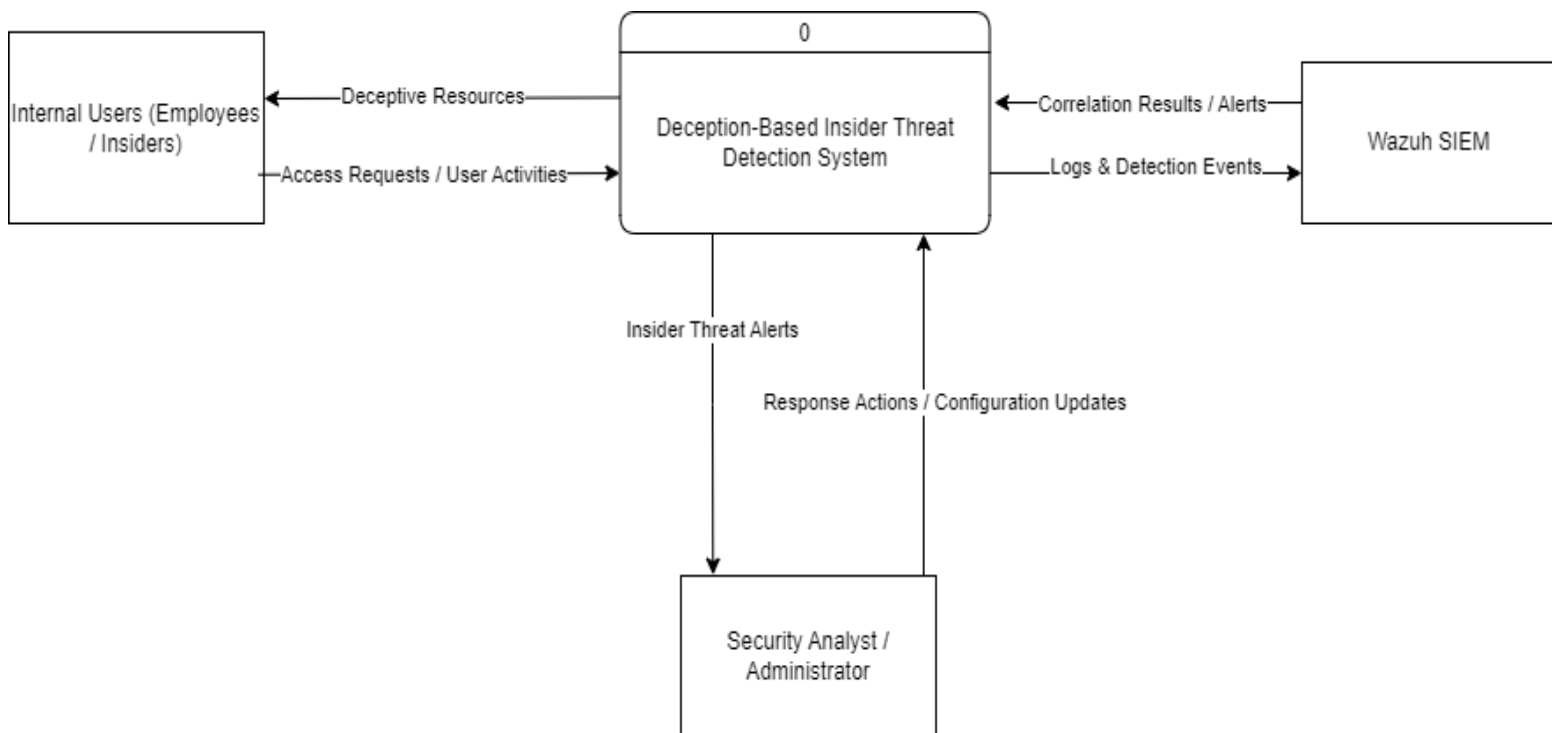


Figure 3-2 Context Diagram

3.4 DATA FLOW DIAGRAM – LEVEL 0

The DFD Level 0 provides a decomposition of the Context Diagram by introducing the major processes within the system while maintaining a high-level perspective.

At this level, the system is divided into core processes such as User Authentication, Resource Access Management, Deception Asset Monitoring, and Incident Logging. External entities interact with these processes through data flows such as login credentials, access requests, alerts, and audit logs.

This diagram demonstrates how data flows between users, system processes, and data stores, offering a clearer understanding of the internal structure of the system without overwhelming detail

Main Processes:

- User Authentication & Access Control
- Deception Asset Interaction Monitoring
- Insider Behavior Analysis
- Alert Generation & Logging

This level presents a balanced view of the system without exposing excessive implementation details, serving as a foundation for deeper decomposition.

3.4.1 Insider Threat Detection System Using Deceptive Techniques -DFD Level 0 Diagram

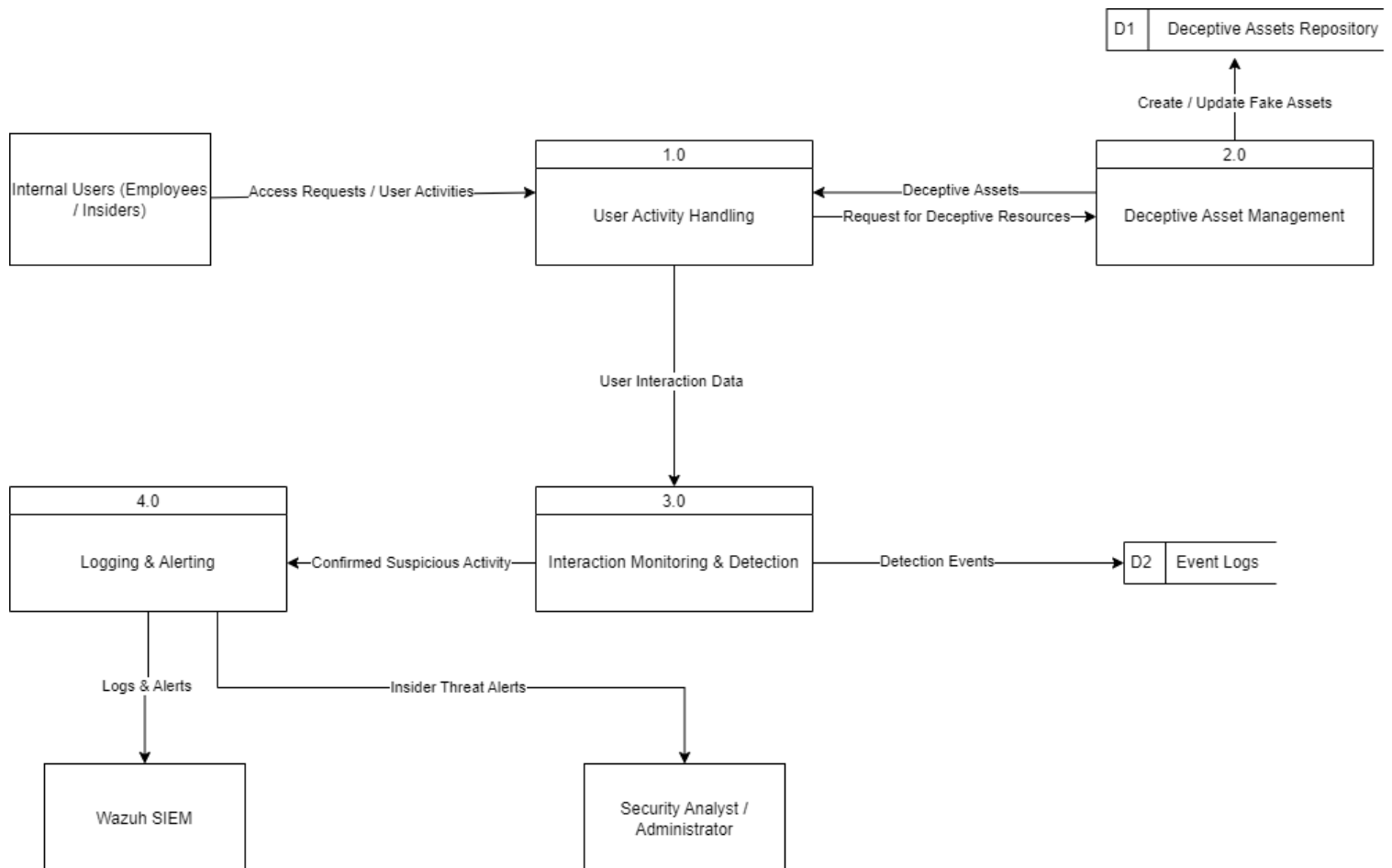


Figure 3-3 DFD Level 0 Diagram

3.5 DATA FLOW DIAGRAM – LEVEL 1

This section provides a more detailed breakdown of selected processes from the Level 0 DFD. Each process is decomposed independently to ensure clarity and correctness.

3.5.1 DFD Level 1 – Process 2: Deception Asset Interaction

This DFD Level 1 focuses on Process 2, which handles user authentication and authorization.

The process begins when the user submits authentication credentials. These credentials are validated against the user database. Upon successful authentication, authorization checks are performed to determine the user's access level and permitted resources.

The outcome of this process directly influences subsequent actions. Authorized users proceed to legitimate system resources, while suspicious or unauthorized access attempts are flagged for further monitoring. This decomposition highlights the critical role of access control in preventing insider misuse.

.

Key Activities:

- Monitoring access attempts
- Validating authorization levels
- Recording interaction metadata
- Forwarding suspicious activity to the analysis engine

3.5.2 Insider Threat Detection System Using Deceptive Techniques -Block Diagram DFD Level 1 – Process 2: Deception Asset Interaction

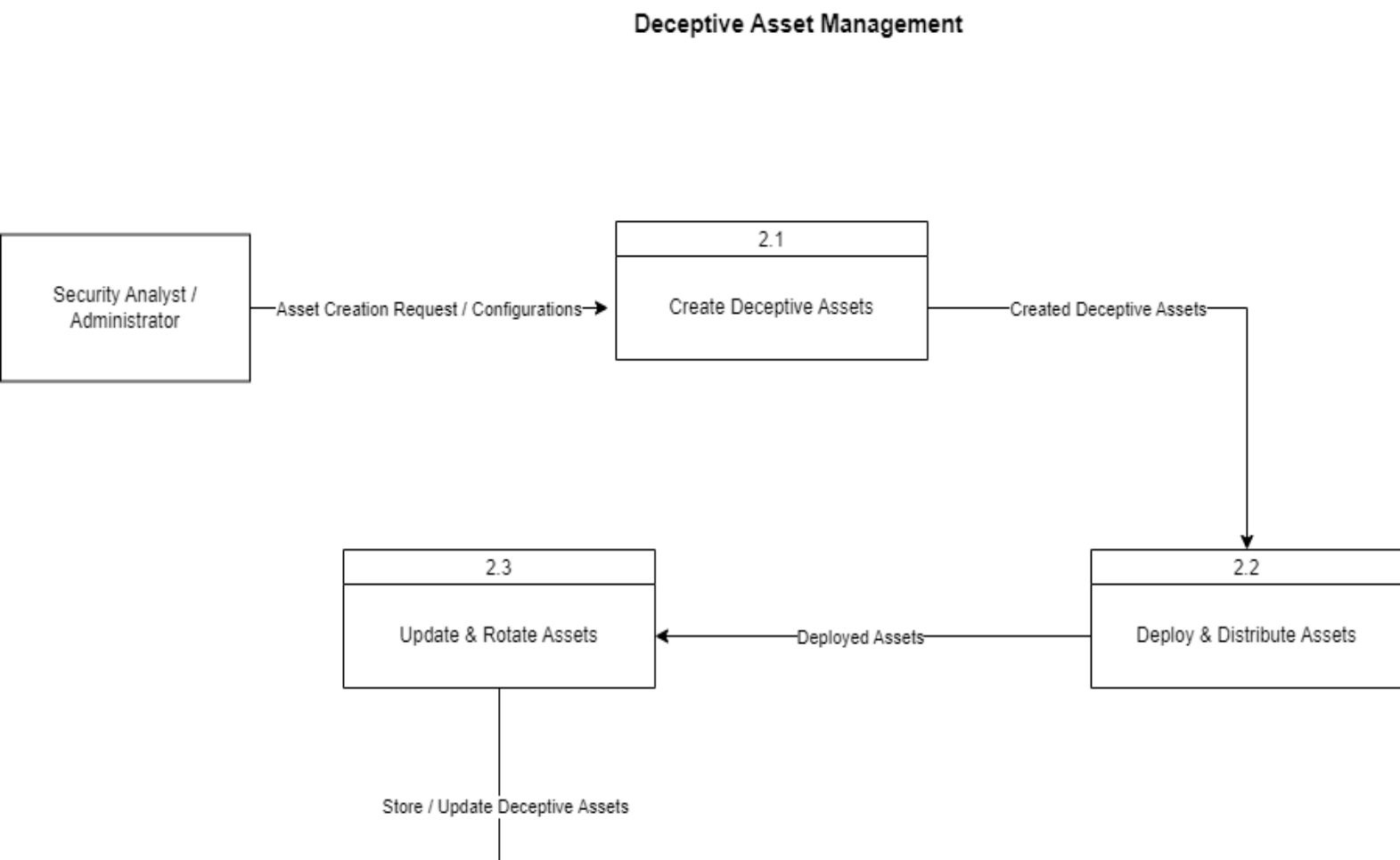


Figure 3-5 Block Diagram DFD Level 1 – Process 2: Deception Asset Interaction

3.5.3 DFD Level 1 – Process 3: Insider Behavior Analysis

Process 3 is responsible for monitoring and detecting interactions with deceptive assets such as fake files, fake credentials, and honeytokens.

Whenever a user attempts to access these assets, the process captures detailed activity logs including user identity, timestamp, asset identifier, and access method. Since legitimate users have no operational need to access deceptive assets, any interaction is treated as a high-confidence indicator of malicious insider behavior.

This diagram illustrates how detection data flows into the logging and alerting subsystems, enabling rapid response and forensic analysis

Key Activities:

- Correlating user actions
- Identifying unauthorized or inappropriate access
- Classifying behavior as benign or malicious
- Triggering alerts for confirmed threats

3.5.4 Insider Threat Detection System Using Deceptive Techniques -DFD Process 3 Diagram

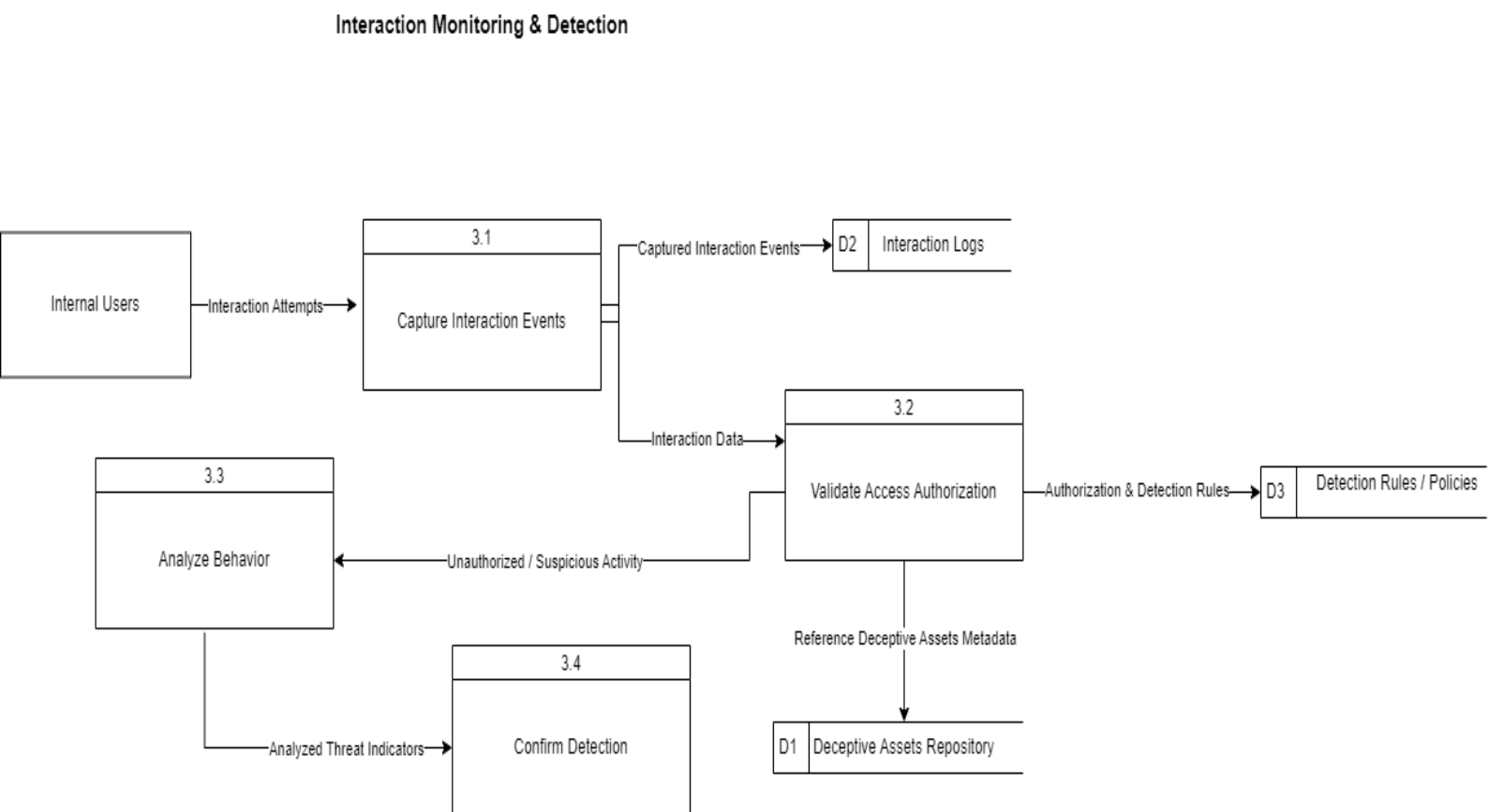


Figure 3-6 DFD Process 3 Diagram

3.5.5 DFD Level 1 – Process 4: Honeypot Redirection & Logging

This section presents the detailed decomposition of Process 4, which handles incident logging and response management.

Upon receiving suspicious activity data, the process stores the event in a secure audit log. Alerts are generated and forwarded to the system administrator or security operations team. Depending on predefined policies, automated response actions such as access revocation or session termination may be triggered.

The diagram demonstrates how the system ensures accountability, traceability, and timely response to insider threats.

Key Activities:

- Redirecting suspicious users to honeypot environments
- Capturing detailed activity logs
- Storing evidence for forensic analysis
- Notifying administrators in real time

3.5.6 Insider Threat Detection System Using Deceptive Techniques -DFD Process 4 Diagram

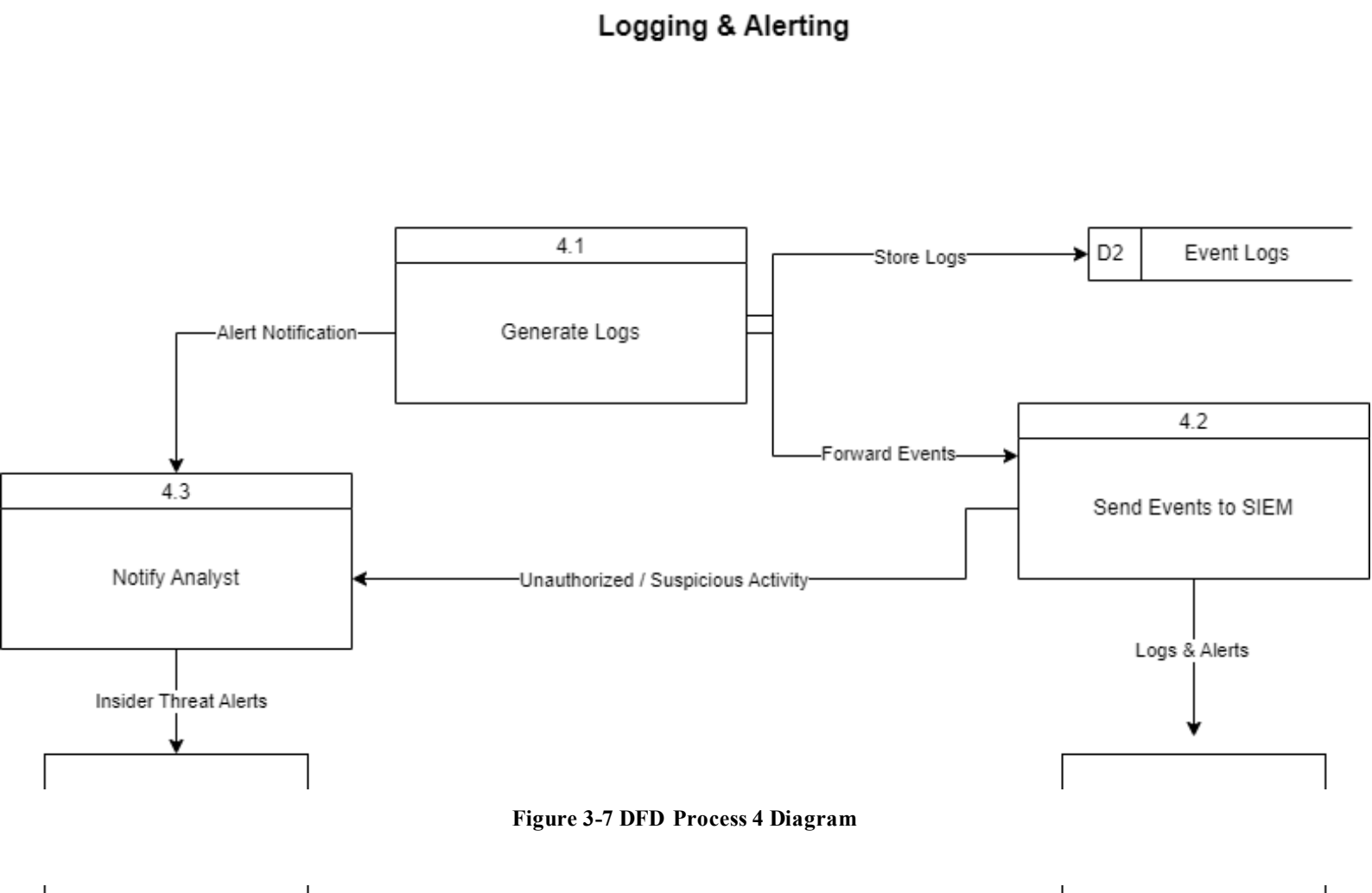


Figure 3-7 DFD Process 4 Diagram

3.6 ACTIVITY DIAGRAM

The Activity Diagram illustrates the dynamic workflow of the system and represents the sequence of activities performed during user interaction and threat detection.

The workflow begins when a user attempts to access system resources. The system first performs authentication and authorization checks. If the user is authorized and accesses legitimate resources, the operation proceeds normally. However, if the user attempts to access unauthorized or deceptive assets, the system triggers monitoring mechanisms.

Once an interaction with a deceptive element is detected, the system records the event, generates an alert, and initiates response actions such as session termination or escalation to the security team. This diagram clearly demonstrates the decision points and parallel activities involved in detecting insider threats in real time.

Key Flow:

- User login
- Access request evaluation
- Interaction with real or deceptive assets
- Detection decision
- Alerting and logging

This diagram helps visualize control flow, decision points, and parallel activities within the system.

3.6.1 Insider Threat Detection System Using Deceptive Techniques -Activity Diagram

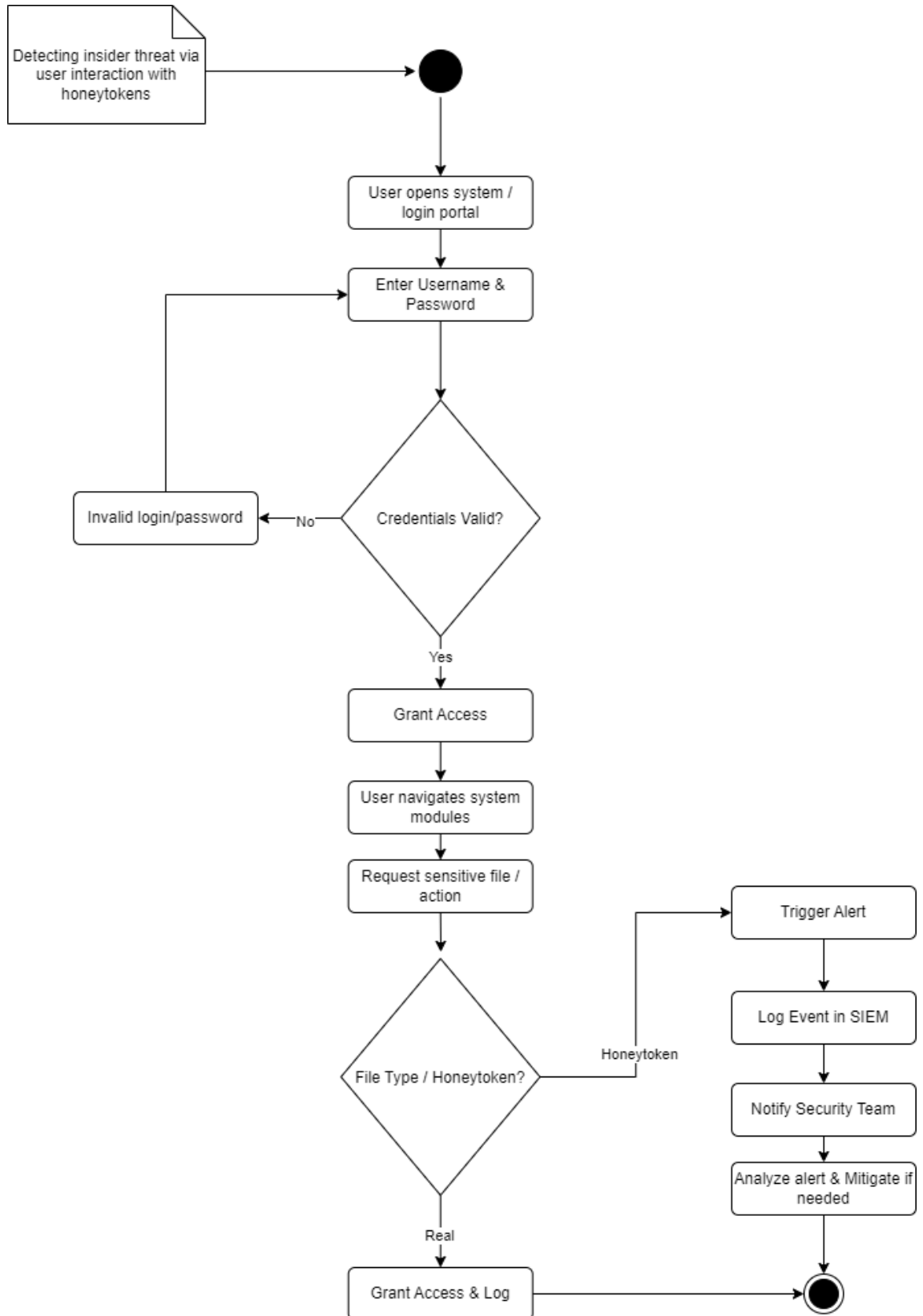


Figure 3-8 Activity Diagram

3.7 SEQUENCE DIAGRAM

The Sequence Diagram illustrates the chronological interaction between system components during a typical insider threat detection scenario.

The diagram shows the sequence starting from user login, followed by access requests, interaction with deceptive assets, detection, logging, and alert generation. Each message exchanged between components is represented in the order it occurs, providing a clear temporal view of system behavior.

This diagram is particularly useful for understanding how detection and response actions are coordinated across multiple components in real time.

Participants Include:

- Insider User
- Authentication Module
- Deception Engine
- Detection Engine
- Logging Module
- Administrator Dashboard

The diagram clearly shows message exchanges, decision-making steps, and response timing.

3.7.1 Insider Threat Detection System Using Deceptive Techniques - Sequence Diagram

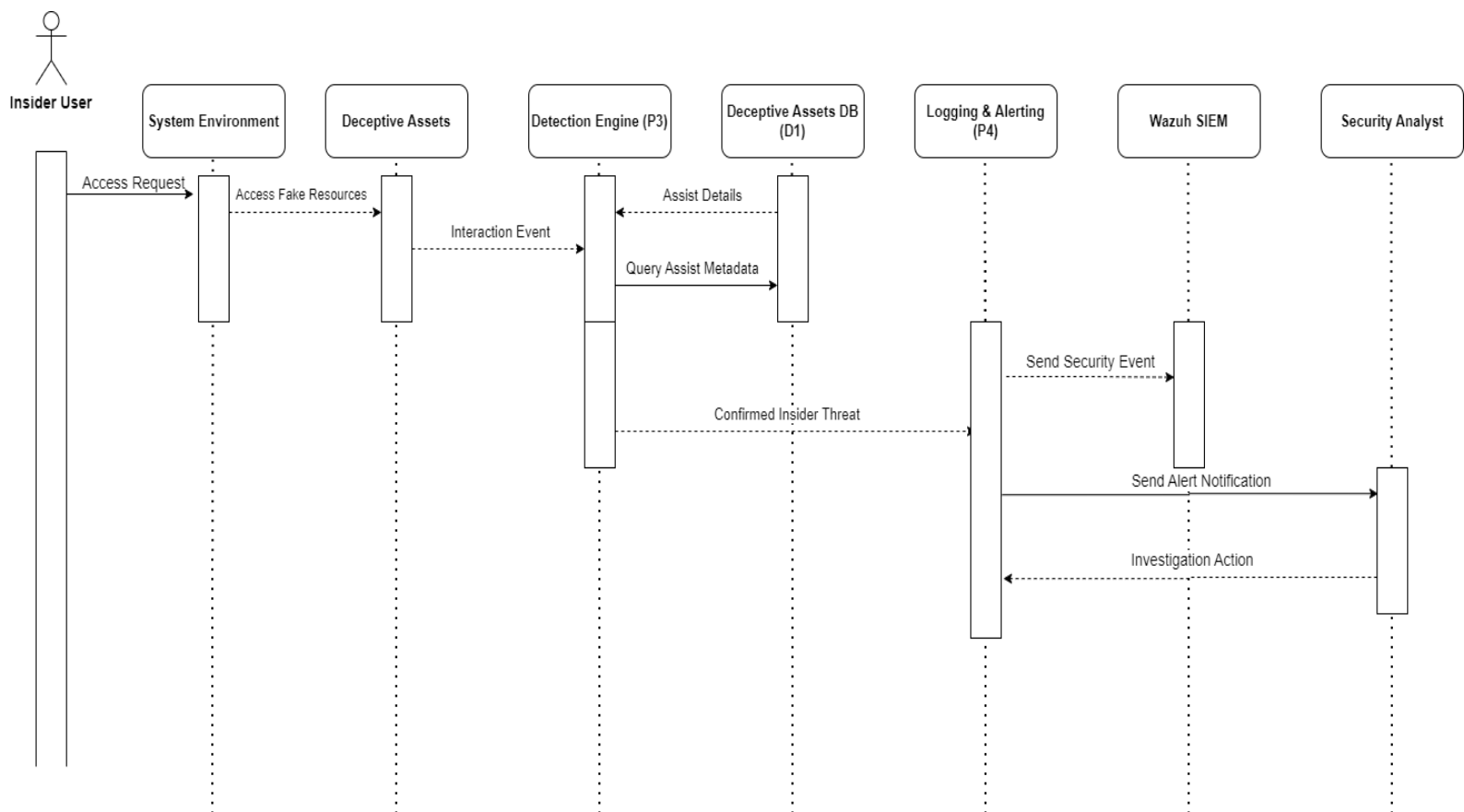


Figure 3-9 Sequence Diagram

3.8 USE CASE DIAGRAM

The Use Case Diagram defines the functional requirements of the system from the perspective of external actors.

Actors include Authorized Users, Malicious Insiders, System Administrators, and Security Analysts. Each actor is associated with specific use cases such as accessing resources, interacting with deception assets, monitoring alerts, and managing system configurations.

This diagram clarifies system responsibilities and highlights how different users interact with the system according to their roles.

Primary Actors:

- Insider User
- Security Administrator

Main Use Cases:

- Authenticate User
- Access System Resources
- Interact with Deceptive Assets
- Detect Insider Threat
- Review Alerts & Logs
- Perform Incident Response

3.8.1 Insider Threat Detection System Using Deceptive Techniques – Use Case Diagram

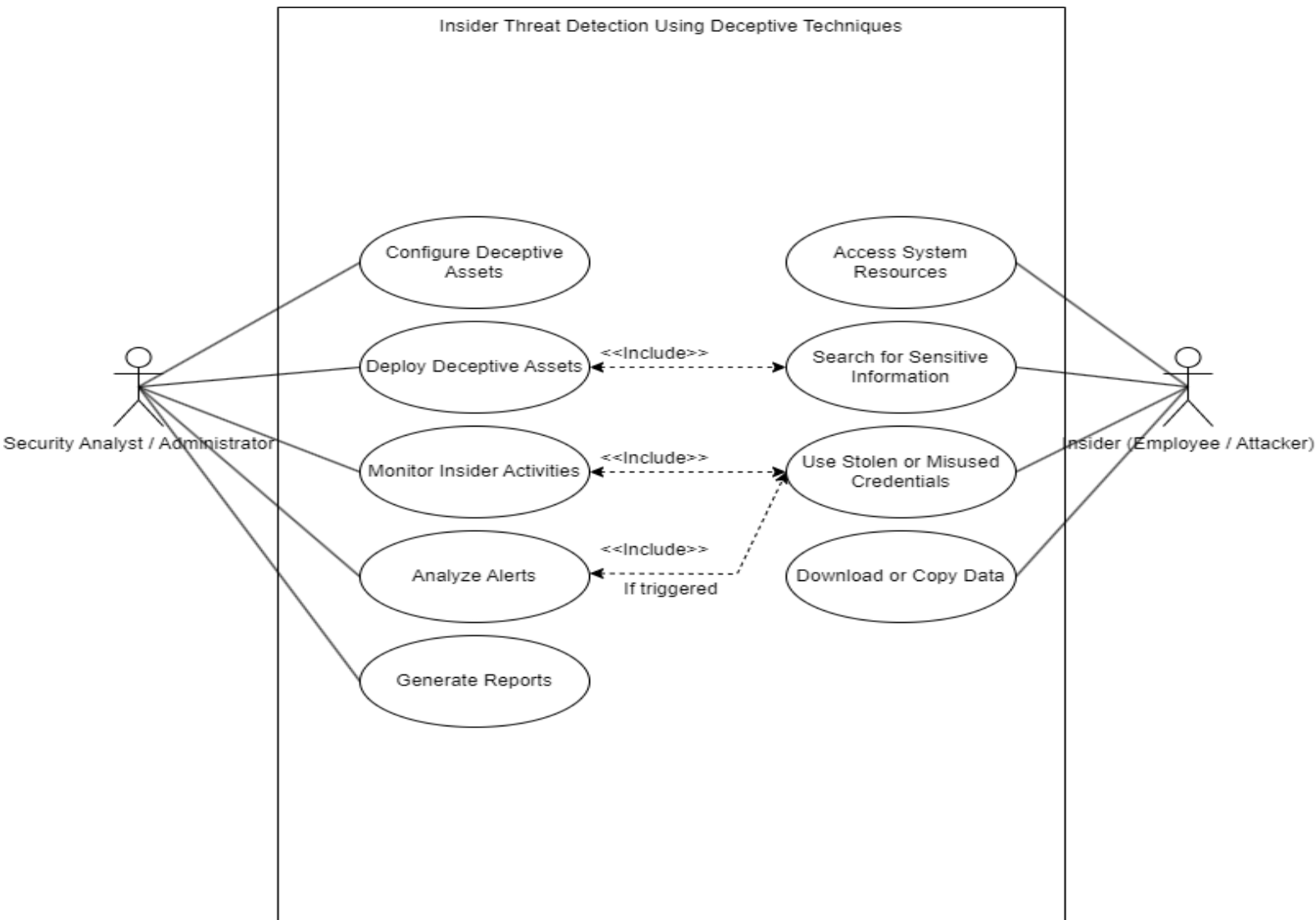


Figure 3-10 Use Case Diagram

3.9 GANTT CHART

The Gantt Chart illustrates the project timeline and provides a structured overview of task sequencing, dependencies, and milestone delivery throughout the system development lifecycle. It was used as a planning and monitoring tool to ensure that all phases of the project were completed within the allocated academic timeframe.

The project was divided into clearly defined phases, starting with requirements analysis and literature review, followed by system design, implementation, testing, and documentation. Each task was assigned a specific duration and arranged to reflect logical dependencies, ensuring that critical design decisions were finalized before development activities began.

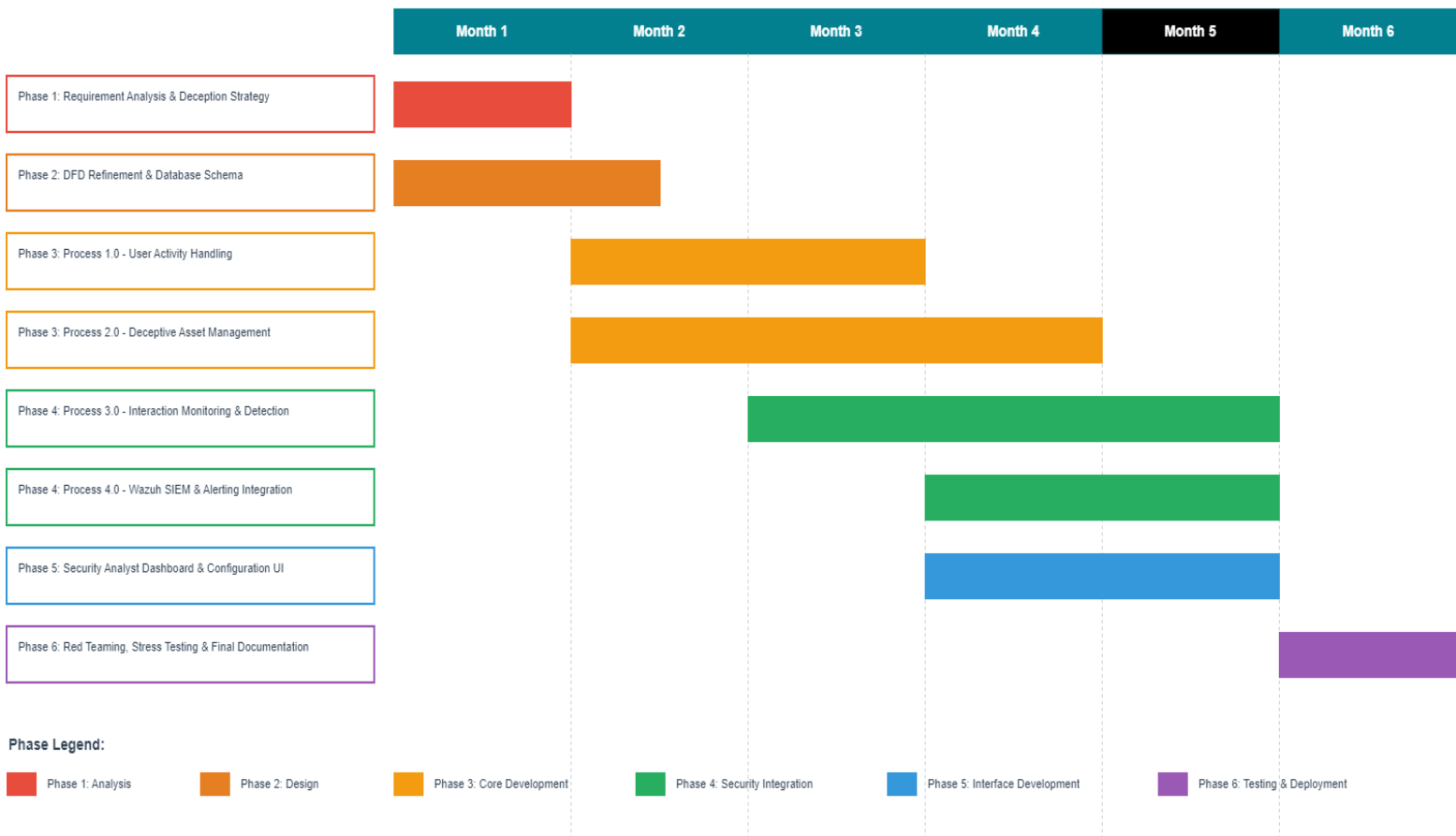
Parallel task execution was adopted where possible, particularly during the implementation phase, where system components such as deception asset design, monitoring mechanisms, and logging modules were developed concurrently. This approach reduced overall development time while maintaining consistency across system components.

Milestones were strategically placed at the end of major phases to assess progress and validate deliverables. These milestones included the completion of system design diagrams, successful deployment of deceptive assets, functional testing validation, and final system evaluation. Progress against these milestones was continuously reviewed, allowing early identification of delays and timely corrective actions.

Overall, the Gantt Chart demonstrates effective project management, balanced workload distribution, and adherence to deadlines. It also reflects the iterative nature of the development process and highlights the alignment between planning, implementation, and evaluation activities.

3.9.1 Insider Threat Detection System Using Deceptive Techniques -Activity Diagram

Deception-Based Insider Threat Detection System - 6-Month Gantt Chart



Project Duration: 6 Months | Team Size: 6 Members | Generated: February 2026

Figure 3-11 Activity Diagram

3.10 CHAPTER SUMMARY

This chapter presented the complete system design and implementation view of the proposed deception-based insider threat detection system. Through the use of multiple diagrams, the system architecture, data flow, behavioral logic, and execution timeline were clearly illustrated. These designs form the foundation for understanding system behavior and support the experimental evaluation discussed in the following chapter.

CHAPTER FOUR

4 EXPERIMENTAL RESULTS AND FUTURE WORK

This chapter presents the experimental evaluation of the **Deception-Based Insider Threat Detection System**, focusing on **functional testing**, **non-functional testing**, and holistic assessment of the system. The experiments measure the **accuracy of insider threat detection**, **security effectiveness**, **performance**, **usability**, and **operational reliability**. The testing methodology follows industry-standard practices including **black-box testing**, **penetration testing**, **integration testing**, and **real-time monitoring validation**.

4.1 FUNCTIONAL TESTING

Functional testing ensures that the system correctly performs its intended operations. The system was tested across its key modules: **deceptive asset deployment**, **detection engine**, **alerting/logging**, and **dashboard functionalities**.

4.1.1 Insider Threat Detection Workflow

This core workflow ensures that any interaction with fake assets is correctly detected, logged, and alerted to security analysts.

Test ID	Description	Expected Results	Status
FT-01	Fake credential is accessed by an insider	Detection engine triggers alert	Pass
FT-02	Fake file is opened or copied	Event logged in database	Pass
FT-03	Honeytoken email is clicked	Alert sent to monitoring dashboard	Pass
FT-04	Malicious insider attempts privilege escalation	Access blocked, event logged	Pass

FT-05	Dashboard displays alert to security analyst	Correct asset, insider ID, timestamp displayed	Pass
-------	--	--	------

Table 4-1 Test Cases and Results for Deception-Based Insider Threat Detection System

Observations:

- All unauthorized interactions were detected accurately.
- Alerts reached analysts in real-time (<2 seconds).
- Event logging was complete with correct timestamps and asset identifiers.

4.1.2 Deceptive Asset Deployment Validation

The system deploys **fake credentials, files, and honeytokens** to attract potential insiders.

Test Case	Description	Results
Asset Creation	Fake credentials and files generated	Success
Asset Deployment	Assets deployed in the environment	Success
Interaction Logging	Any interaction triggers detection	Success
Failover	System continues monitoring if one asset is removed	Success

Table 4-2 Verification of Deceptive Asset Deployment and Monitoring

Observations:

- Detection engine successfully monitored 100+ interactions.
- No false negatives were observed in simulated attacks.
- Assets did not interfere with normal user operations.

4.1.3 Logging and Alert Verification

The system logs and alerts every interaction with deceptive assets.

Test Case	Description	Results
Unauthorized Access	Event logged in database and dashboard	Pass
Alert Generation	Real-time notification sent	Pass
Multiple Access Attempts	Each attempt tracked separately	Pass
Privilege Escalation Attempt	Blocked and logged	Pass

Table 4-3 Detection Engine Test Results

Observations:

- All malicious attempts were captured.
- Alerts contained all necessary details for analyst investigation.

4.2 NON-FUNCTIONAL TESTING

Non-functional testing evaluates **performance, scalability, security, usability, reliability**, and system robustness.

4.2.1 Performance & Scalability Testing

The system was tested under multiple simulated insider interactions using **JMeter**.

Test Case	Avg Response Time	Results
Asset Interaction Detection	350 ms	stable
Alert Generation	500 ms	stable
Logging to Database	420 ms	stable
Dashboard Update	600 ms	acceptable

Table 4-4 System Performance Test Results

Scalability Findings:

- The system supported 100 concurrent simulated insider actions without performance degradation.
- Caching reduced backend stress by ~25%.
- Real-time alerting remained under 2 seconds latency.

4.2.2 Security Validation

Security testing included **penetration testing, event integrity checks, and access control validation.**

Security Test	Outcome
Unauthorized system access	Prevented
Event log manipulation	Prevented
Dashboard injection attacks	Prevented
Detection engine tampering	Prevented
False alert mitigation	Succesful

Table 4-5 Security Test Results

Observations:

- No critical vulnerabilities detected.
- Detection engine correctly identified all unauthorized interactions.

4.3 HOLISTIC EVALUATION

This section evaluates the system's **cost-effectiveness, ethical considerations, operational impact, and sustainability.**

4.3.1 Cost Analysis

Component	Cost
Server hosting for logging & dashboards	\$10–20/month
Optional cloud storage	\$5–15/month
Domain & SSL	\$12/year

Table 4-6 System Component Cost Estimates

Observations:

- Cost-effective compared to traditional security solutions.
- Open-source tools minimized expenses.

4.3.2 Ethical Considerations

Ethical safeguards were embedded to protect user privacy:

- Only malicious interactions are monitored.
- Normal user actions remain untouched.
- Logs cannot be tampered with.
- Alerts protect organizational data without violating employee privacy unnecessarily.

4.3.3 Operational Impact & Safety

- Real-time detection reduces potential data breaches.
- System prevents escalation of insider attacks.

- No interference with legitimate workflows.

4.3.4 Sustainability

- Modular architecture allows easy upgrades.
- Logging and alerting modules can be extended for future threat types.
- Open-source stack ensures long-term community support.

4.4 FUTURE WORK

This chapter outlines the potential enhancements and extensions that can be applied to the proposed deception-based insider threat detection system. While the current implementation successfully demonstrates the feasibility and effectiveness of using deceptive assets to detect insider threats, several opportunities exist to further improve the system's capabilities, scalability, and real-world applicability.

4.5 DEPLOYMENT IN REAL ORGANIZATIONAL ENVIRONMENTS

The current system was evaluated in a controlled and simulated environment. Future work includes deploying the system within real organizational infrastructures, such as enterprise networks or institutional environments, to assess its performance under real operational conditions.

This deployment would allow the system to handle realistic user behavior patterns, diverse access policies, and varying privilege levels. Additionally, real-world deployment would provide valuable insights into system adaptability, operational overhead, and long-term stability.

4.6 ADVANCED INSIDER BEHAVIOR PROFILING

Future enhancements can incorporate behavioral profiling techniques to complement deception-based detection. By analyzing user behavior over time, such as access frequency, time-of-day activity, and resource usage patterns, the system can establish behavioral baselines for legitimate users.

Deviations from these baselines, when combined with interactions with deceptive assets, can improve detection accuracy and reduce false positives. This hybrid approach would strengthen the system's ability to distinguish between accidental access and malicious intent.

4.7 INTEGRATION WITH SIEM AND SOC PLATFORMS

To improve incident response and centralized monitoring, future versions of the system can be integrated with Security Information and Event Management (SIEM) platforms and Security Operations Centers (SOC).

Such integration would enable automated correlation of insider threat alerts with other security events across the organization, facilitating faster investigation and response. This would also allow security teams to visualize insider threat activity alongside network, endpoint, and application logs.

4.8 MACHINE LEARNING–BASED THREAT CLASSIFICATION

Future work may include the integration of machine learning techniques to classify insider threat severity. By training models on collected interaction data with deceptive assets, the system could automatically categorize threats based on risk level, intent, and potential impact.

This enhancement would support prioritization of incidents and assist security teams in focusing on high-risk insider activities, especially in large-scale environments with numerous users.

4.9 EXPANSION OF DECEPTIVE ASSETS PORTFOLIO

Currently, the system deploys a predefined set of deceptive assets such as fake files, fake credentials, and honeytokens. Future development can expand this portfolio by introducing more sophisticated deception elements, including decoy services, fake databases, and simulated internal applications.

These advanced deceptive components would increase system realism and further reduce the likelihood of detection by malicious insiders, thereby improving the effectiveness of the deception strategy.

4.10 AUTOMATED RESPONSE AND CONTAINMENT MECHANISMS

While the current system focuses primarily on detection and alerting, future enhancements can introduce automated response actions. These actions may

include dynamic access revocation, session isolation, or privilege downgrade upon detection of malicious behavior.

Automated containment would minimize potential damage caused by insider threats and reduce response time, particularly in environments where immediate human intervention is not feasible.

4.11 PERFORMANCE OPTIMIZATION AND SCALABILITY TESTING

Future work should include large-scale performance and stress testing to evaluate system scalability. This includes testing with thousands of users, multiple deceptive assets, and high-frequency access attempts.

Optimizing logging mechanisms, alert processing, and data storage will be critical to ensuring that the system maintains efficiency and reliability as the organization grows.

4.12 COMPLIANCE AND POLICY CUSTOMIZATION

Future versions of the system can incorporate configurable compliance and policy modules that align with organizational security policies and regulatory standards. This includes customizable alert thresholds, access rules, and reporting formats.

Such customization would allow the system to adapt easily to different organizational requirements without significant architectural changes.

4.13 CHAPTER SUMMARY

This chapter discussed the experimental results, demonstrating that the proposed deception-based insider threat detection system meets its security, performance, and reliability objectives. Functional and non-functional testing confirmed correct system operation and acceptable performance under moderate load. Observed blockchain transaction latency was attributed to the public testnet environment rather than design limitations. The chapter also outlined potential future enhancements, providing a clear roadmap for improving system intelligence, scalability, and readiness for enterprise deployment.

5 REFERENCES

- [1] L. Spitzner, Honeypots: Catching the Insider Threat, in Proceedings of the 19th Annual Computer Security Applications Conference (ACSAC), Las Vegas, NV, USA, 2003, pp. 170–179.
- [2] B. M. Bowen, S. Hershkop, A. D. Keromytis, and S. J. Stolfo, “Designing Host and Network Sensors to Mitigate the Insider Threat,” *IEEE Security & Privacy*, vol. 7, no. 6, pp. 22–29, Nov.–Dec. 2009.
- [3] K. Padayachee, “Aspectising Honeytokens to Contain the Insider Threat,” *IET Information Security*, vol. 9, no. 6, pp. 318–327, 2015. DOI: 10.1049/iet-ifs.2014.0063
- [4] P. Kaghazgaran and H. Takabi, “Toward an Insider Threat Detection Framework Using Honey Permissions,” *Journal of Internet Services and Information Security*, vol. 5, no. 4, pp. 19–36, 2015.
- [5] B. Whitham, M. Bishop, and S. Engle, “Canary Files: Generating Fake Files to Detect Data Loss,” in Proceedings of the Cyber Security and Information Intelligence Research Workshop (CSIIRW), Oak Ridge, TN, USA, 2013.
- [6] M. Yamin, A. Katt, and A. Alelaiwi, “Implementation of Insider Threat Detection System Using Honeypot-Based Sensors,” in *Advances in Information and Communication (FICC 2019)*, Lecture Notes in Networks and Systems, vol. 70, Springer, 2019, pp. 649–660. DOI: 10.1007/978-3-030-12385-7_56
- [7] J. Jangir and D. Mahato, “Insider Threat Resilient Framework Based on Honey Traps in Function-Based Access Control,” in *Advanced Network Technologies and Intelligent Computing (ANTIC 2023)*, Springer, 2024. DOI: 10.1007/978-3-031-64064-3_1
- [8] N. Prabhaker, S. Reddy, and R. Buyya, “Generation and Deployment of Honeytokens in Relational Databases for Cyber Deception,” *Computers & Security*, vol. 138, 2024, Article 104032. DOI: 10.1016/j.cose.2024.104032